

Práctico (Final): MLOps Local con Monitoreo de Data Drift

Magíster en Ciencia de Datos — Tópicos en Data Science II

Modalidad: Grupal de 2 a 4 integrantes (si son más notificarme por correo)

Evaluación: Este trabajo reemplaza la **Prueba de Cátedra** y el **Examen** del curso.

Formato de entrega: Repositorio de código + informe ejecutivo (PDF) + defensa oral grupal (10 min + 5 min de preguntas).

1. Contexto y motivación

En producción, un modelo de Machine Learning no falla necesariamente porque el código se rompe: falla porque el mundo cambia y los datos que recibe dejan de parecerse a los datos con los que fue entrenado (**data drift**) o porque la relación entre variables y objetivo cambia (**concept drift**). Sin un sistema de monitoreo, este deterioro es silencioso hasta que el impacto de negocio ya es inevitable, evidente y costoso.

Este práctico integra todos los recursos y aprendizajes obtenidos durante el curso (RAA1-RAA5): deberán elegir e identificar una problemática real, realizar un EDA, construir un pipeline de modelado, evaluar la existencia de fuga de datos, empaquetarlo como servicio, e implementar un sistema que **detecta cuándo el modelo empieza a fallar por drift** y definir qué hacer al respecto.

2. Resultados de aprendizaje evaluados

RAA	Cómo se evalúa en este práctico
RAA1 — Ciclo de vida de proyectos de Ciencia de Datos	Estructura general del proyecto: problema → datos → modelo → servicio → monitoreo → acción
RAA2 — Exploración de datos	Fase 1 (EDA), reutilizando criterios de los prácticos 3 y 4
RAA3 — Desarrollo de soluciones reales en contexto industrial	Fases 2 y 3 (modelado sin fuga de datos, empaquetamiento, servicio de inferencia)
RAA4 — Liberación, monitoreo y mejora continua	Fases 4 y 5 (simulación de producción, drift, plan de reentrenamiento)
RAA5 — Identificar problemáticas abordables con CD	Fase 0 (elección y justificación de negocio de la problemática)

3. Elección de la problemática (Fase 0)

Deben elegir **una** problemática real, puede ser tambien basada en datos públicos (Kaggle u otra fuente abierta), justificando en 1 párrafo por qué es un caso donde el drift es relevante (estacionalidad, cambios de comportamiento, shocks externos, por darles ejemplos).

Sugerencias : Para no partir de cero pueden reusar EDA/pipelines de prácticos anteriores basado en sus respuestas entregadas durante los practicos, no esta permitido usar el mismo caso de ejemplo de los practicos anteriores. En esos casos anteriores les explico en la siguiente tabla porque son utiles los data set anteriores:

Dataset	Por qué funciona para drift	Esfuerzo
Olist E-commerce (ya usado en prácticos 03-04)	Tiene timestamps reales 2016-2018; se puede definir ventana “referencia” vs “producción” con corte temporal real (ej. predecir atraso de entrega o rating de reseña)	Bajo (ya conocen los datos)
Rossmann Store Sales / Walmart Sales Forecasting	Series temporales con estacionalidad y eventos (feriados, promociones) — drift natural	Medio
Telco Customer Churn / Credit Risk (practico 05)	Sin eje temporal fuerte → el drift se simula con muestreo sesgado o perturbación de features (técnica igualmente válida y muy usada en la industria para testear sistemas de monitoreo)	Bajo-Medio

4. Requisitos técnicos (stack liviano — sin Docker ni servidores dedicados)

El objetivo es que el sistema completo se levante en un notebook con recursos limitados, con **un solo comando o script**, sin infraestructura cloud ni contenedores obligatorios.

Componente	Herramienta sugerida	Notas
Entrenamiento y versionado	<code>scikit-learn / xgboost + joblib</code> (o <code>mlflow</code> en modo local <code>file:./mlruns</code> , opcional)	No se exige un servidor MLflow; el tracking local a archivo ya cumple el objetivo
Servicio de inferencia	<code>FastAPI + uvicorn</code> corriendo local (<code>uvicorn app:app</code>)	Una sola línea para levantar; no requiere contenedor
Simulación de “producción”	Split temporal o muestreo sesgado del dataset, alimentado al servicio en lotes/streaming simulado	Debe generar al menos 3-4 “ventanas” de datos con drift creciente
Detección de drift	Estadísticos propios (PSI, KS-test, χ^2 para categóricas) implementados con <code>scipy/numpy</code> , o librería <code>evidently</code> (<code>pip install</code> , sin servicio adicional)	Se exige justificar el umbral elegido (ej. $\text{PSI} > 0.2$ = drift significativo)
Dashboard de monitoreo	<code>Streamlit</code> mostrando: performance del modelo en el tiempo, métricas de drift por feature, alertas	Un comando: <code>streamlit run dashboard.py</code>
Registro de predicciones	CSV/SQLite local	No se requiere base de datos productiva

Docker Compose, Kubernetes, servicios cloud (AWS/GCP/Azure) o herramientas de pago quedan explícitamente fuera de alcance. No suman puntaje extra y penalizan si comprometen que el script pueda ejecutar el proyecto en local o su propia máquina en menos de 10 minutos.

5. Fases del trabajo

1. **Fase 0 — Problemática:** elección y justificación de negocio (RAA5).
2. **Fase 1 — EDA:** análisis univariado/multivariado, detección de fuga de datos potencial (RAA2).
3. **Fase 2 — Modelado:** pipeline sin fuga de datos, validación robusta, empaquetado reproducible (RAA3).
4. **Fase 3 — Servicio:** API de inferencia local + registro de cada predicción con sus inputs.
5. **Fase 4 — Monitoreo de drift:** simulación de ventanas temporales de “producción”, cálculo de métricas de drift por variable y del target/performance, dashboard.

6. **Fase 5 — Plan de acción:** cuando el drift supera el umbral definido, ¿qué hace el sistema? (alerta, trigger de reentrenamiento automático o semi-automático, rollback a versión anterior). No es obligatorio automatizar el reentrenamiento completo, pero sí **documentar y dejar el gatillo implementado** (aunque sea un script que el usuario ejecuta al recibir la alerta).
-

6. Entregables

1. Repositorio (Git o zip) con README.md guía de instalación y para su ejecución en < 10 min.
 2. Notebook o script de EDA y justificación del modelo.
 3. Script del servicio de inferencia + del módulo de detección de drift.
 4. Dashboard funcional (Streamlit).
 5. Informe ejecutivo (maximo 3 páginas, para audiencia de negocio, no técnica): problemática, riesgo de no monitorear drift, resultados, plan de acción.
 6. Defensa oral grupal (maximo 10 min + ronda de preguntas maximo 5 min): demo en vivo del sistema detectando drift + preguntas al grupo.
-

7. Rúbrica de evaluación (100 puntos)

Criterio	Ponderación	RAA	Indicadores
Problemática y justificación de negocio	5%	RAA5	Claridad del caso de uso, relevancia del drift para ese dominio
EDA y calidad de datos	10%	RAA2	Profundidad del análisis, detección de anomalías/fuga potencial
Modelado y buenas prácticas	10%	RAA3	Sin fuga de datos, validación correcta, métricas apropiadas al problema
Servicio de inferencia local	10%	RAA3/RAA4	Funciona con un comando, registra predicciones, reproducible

Criterio	Ponderación	RAA	Indicadores
Sistema de monitoreo de drift	20%	RAA4	Métricas de drift correctamente calculadas e interpretadas, umbrales justificados, dashboard claro
Plan de acción ante drift	5%	RAA4	Gatillo de alerta/reentrenamiento documentado e implementado (aunque sea semi-manual)
Informe ejecutivo y defensa oral	40%	Todos	Comunicación clara a audiencia no técnica, comprensión individual demostrada en defensa

Criterio de corte: un proyecto que no se pueda ejecutar localmente en máquina en menos de 10 minutos siguiendo el `README.md` no puede obtener más de 60/100, independiente de la calidad del análisis, la ejecutabilidad local es un requisito explícito del enunciado.

Uso de IA: Para este trabajo pueden utilizar la IA como agente auxiliar como apoyo en *vibe coding*. Pero no está permitido usar el agente para generar el informe final, recomiendo que lo escriban ustedes, ya que deben defender su proyecto y demostrar su comprensión.